

• ACM ALEXANDRIA STUDENT CHAPTER

Agentic AI Bootcamp & Hackathon

Two days of building with AI agents. Then a hackathon to prove it.

Sep 7 · **3 to 5 PM** · Bootcamp Day 1

Sep 8 · **3 to 6 PM** · Bootcamp Day 2

Hackathon · **Date & time TBD**

No prior AI experience needed · with Yusuf Morsi

• MEET YOUR INSTRUCTOR

Yusuf Morsi

AI/ML engineer · 8× hackathon winner

What he does

- Software Engineer at Cisco, working on ML and RAG systems and large-scale data pipelines
- Works with Prokuro.ai on the side, building agentic AI for supply chain
- Production AI systems, multi-agent architectures, LLM/RAG

What he did

- 8× hackathon winner, including Cambridge, Stanford, and UCLA
- M.S. Machine Learning & Data Science, University of California, San Diego
- B.S. Electrical Engineering, University of California, San Diego



Follow Prokuro.ai

linkedin.com/company/prokuro



Connect on LinkedIn

linkedin.com/in/yusufmorsi

What is an LLM?

Large Language Model: a next-word predictor, not a mind.

LIVE DEMO: ask it something, then make it hallucinate

1

Your words, the prompt

2

Guess next word, then the next,
and the next

3

A full answer that looks fluent,
but may still be wrong

Strengths

- Explaining and drafting in plain language
- Pattern-matching, including code
- Fast first versions

Limits

- Can be confidently wrong
- Does not know your files unless you give them
- No live knowledge unless connected

What is a coding agent?

Plain chat

- You paste code in and copy code out
- It replies in a box
- You run everything yourself
- One shot, then you start again

Coding agent

- Reads and writes files in your project
- Runs commands, tests, installs, git
- Sees errors and tries again
- You steer. It loops.

Chat answers. An agent **works in the repo**.

What are Agent Skills?

A Skill is a reusable playbook the agent can load for a specific kind of task, instead of you re-explaining it every time.

The problem

Re-explaining "how we do slides" or "our code style" in every single prompt wastes time and gets inconsistent results.

A Skill

A folder of instructions for one job, like "make a PDF" or "follow our PR checklist." The agent loads it automatically when relevant.

Why it matters

Consistent results, less repeated typing, easy to write your own for something you do often.

Tomorrow: if you find yourselves repeating the same instructions to the agent, that's a sign to turn it into a Skill.

Intro to Claude Code

Anthropic's coding agent. Lives in the terminal and your editor.

LIVE DEMO: install it and run one real task on screen

What it is

- You describe the task in English
- It edits the project in front of you
- You approve, reject, or redirect

Install and run

```
curl -fsSL https://claude.ai/install.sh | bash  
cd your-project  
claude
```

```
Windows: irm https://claude.ai/  
install.ps1 | iex
```

Setting up your tools

Two options. Pick one before tomorrow. Both work with everything we teach.

Claude Code (recommended)

- Free to install, terminal + editor based
- Our sponsor's AI credits work directly with it
- Setup: `curl -fsSL https://claude.ai/install.sh | bash`

Cursor (alternative)

- A full code editor with AI built in
- Free "Hobby" tier exists, but with limited agent requests/month
- Setup: download from cursor.com, sign in, start a project

If you're not sure which to pick, use Claude Code, since the sponsored credits are set up for it directly. Tip: `/model` and `/effort` tune power vs. cost, and `/permissions` lets you allowlist trusted actions instead of clicking yes to everything.

• DAY 1 · GROUNDING ANSWERS IN YOUR DATA

What is RAG?

Retrieval-Augmented Generation: look it up, then answer. Same idea powers coding agents, since giving sources beats relying on memory.

LIVE DEMO: ask a model something not in its training vs. something you feed it via a doc

Feed the base

Your notes, README, API docs, or PDFs. The agent reads them first.

Retrieve & generate

It pulls the right pages, then answers or writes code using them instead of guessing.

You check

Does the answer match the source? If not, point it at the file.

Prompt engineering basics

Better questions, better answers. Four habits we will use tomorrow.

Be clear

Say the goal, the audience, and the format. Vague in, vague out.

Give context

Who it is for, constraints, tools, and what done looks like.

Show examples

One good sample of the style or output beats a long lecture.

Iterate

First try is a draft. Tighten it: shorter, in Arabic and English, add a tiny example. Do not start over, just refine.

• DAY 1 · TRY IT YOURSELF

Live prompting demo

Open Claude or ChatGPT on your own laptop. Paste this. Then change one line and rerun.

```
You are tutoring a first-year CS student.  
Explain what a function is in 4 short sentences,  
using an analogy from daily life.  
Then give one tiny Python example with a comment.
```

- Swap the analogy (bus, kitchen, library card)
- Ask for Arabic and English
- Make it shorter, or add a common mistake

Git and version control

A timeline of your project. Teams need it. Agents need it too.

Why it matters

- Undo a bad agent edit
- See what changed (the diff)
- Share work without emailing zip files

Commands you will use

```
git init
git status
git add .
git commit -m "message"
git log --oneline
```

Commit after each working step, not a 40-file dump. Mess mid-task? Esc Esc or /rewind rolls back fast.

Reviewing AI-generated code

The agent wrote it. You're still responsible for it. A simple checklist before you commit.

Every time, ask:

- Did I actually run it?
- Does the diff touch only what I asked for?
- Would I understand this code in a week?
- Any secrets, keys, or weird new dependencies snuck in?

Red flags

- Changes to files you didn't mention
- Deleted tests or error handling
- Code that "looks right" but you can't explain
- A fix that just silences an error instead of solving it

Rule of thumb: if you can't explain what a change does, don't commit it yet. Ask the agent to explain first.

• DAY 1 WRAP UP

What to expect on Day 2

Sep 8, 3 to 6 PM. Today was concepts. Tomorrow we build, live, one project across three levels of depth.

You'll watch

A real working prototype, built start to finish, using everything from today.

Three levels

Frontend, then backend, then real RAG, each building on the last.

Bring

A laptop with Claude Code or Cursor installed, and a charger.

See you tomorrow. Any questions before then, ask in the group.

• DAY 2 · SEP 8 · SCOPE IT TOGETHER

Worked example: Study Buddy

Plan the prototype

Study Buddy: a tiny app that answers questions using only your own notes. Built live today, in three stages.

Must have

- notes.md, 8-10 facts
- Question box, one page
- Answer + source note

Nice if time

- Highlight matching line
- Arabic + English UI

Out of scope

- Login, cloud, databases
- Anything >2 min to demo

Ask the room: what question should the app answer?

• DAY 2 · TODAY'S PLAN

Three depths, built live

Same Study Buddy, three stages, each one building on the last. Watch closely, take notes, ask questions any time.

LEVEL 1

Frontend

A one-page UI with keyword search over notes.md. No server needed.

LEVEL 2

Backend

A small server that serves notes over an API, with better search ranking.

LEVEL 3

Advanced

Real embeddings-based RAG instead of keyword matching.

Built live today, one instructor laptop, since venue Wi-Fi can't support everyone running agents at once.

• DEEPER LOOK

What is RAG, visually?

Retrieval-Augmented Generation: the model looks something up before it answers, instead of guessing from memory.

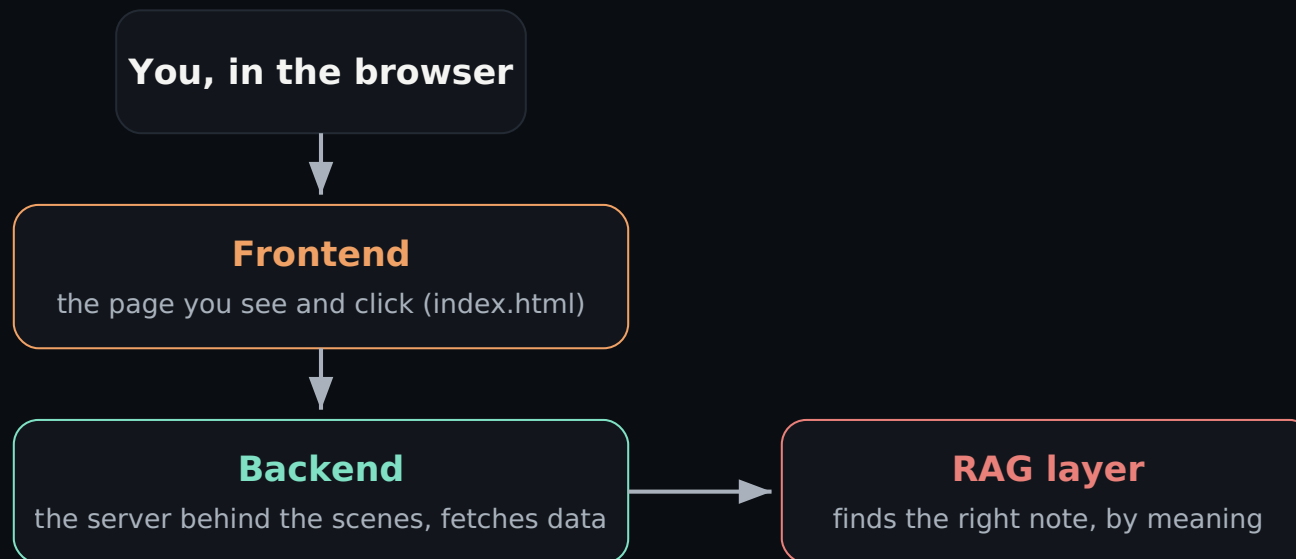


This is exactly what we're about to build with Study Buddy: notes.md as the source, the app retrieves the right part, then answers from it, never from memory alone.

• DEEPER LOOK

How the three pieces fit together

Frontend, backend, and RAG, in plain terms, and how they stack.



Level 1 will just be the frontend, notes read directly.

Level 2 will add the backend in between.

Level 3 will make the backend smart with RAG.

Coding agent workflow

1 · Plan

notes.md + one page

2 · Prompt

One clear task

3 · Review diff

Read it, don't auto-accept

4 · Fix

Paste the error, retry

THEN REPEAT: SMALL LOOPS BEAT GIANT PROMPTS

Good: "notes.md with 8 facts, then a page that searches them." **Weak:** "Build a full AI study platform."

Building Level 1: Frontend

1. **New folder** → `git init` → `claude`
2. **Prompt:** `notes.md` with 8 to 10 real facts (course, campus, a hobby)
3. **Prompt:** one HTML page, question box, keyword search, show matching note
4. **Test it:** ask something not in the notes. What happens?
5. `git commit -m "Level 1: frontend search working"`

If it's not in `notes.md`, it should say "I don't know," never make something up.

[ASK THE ROOM] one feature to add before we move on

Building Level 2: Backend

1. **Prompt:** add a small local server that serves notes.md over a simple API endpoint
2. **Prompt:** update the page to call that API instead of reading the file directly
3. **Prompt:** improve ranking so the best match comes first, not just the first keyword hit
4. **Test it:** ask a question that matches two notes, check which one wins
5. `git commit -m "Level 2: backend API + better ranking"`

Notice you're giving the agent similar instructions again? That's a candidate for a Skill from Day 1.

[ASK THE ROOM] would your hackathon idea need a backend like this?

Building Level 3: Advanced (RAG)

1. **First, break it:** ask Level 2 a paraphrased question, no exact keyword match, watch it fail
2. **Predict:** ask the room why it failed, before you fix it
3. **Prompt:** replace keyword search with real embeddings-based retrieval, matching by meaning, not exact words
4. **Optional:** split into two agents, one retrieves the notes, one drafts the answer
5. **Re-test:** same paraphrased question, now it finds the right note
6. `git commit -m "Level 3: real RAG retrieval"`

This is the gap between "it looks like RAG" and "it actually retrieves by meaning." Worth the extra step.

Anti-patterns and hallucination

Fluent is not the same as true. Watch across all three levels.

What goes wrong

- Answers using facts not in notes.md
- Edits 12 files for one ask
- Looks done, search never runs

How we catch it

- Ask a question you know is missing
- Read the diff before accepting
- Commit only after a working demo

Rule: if the source is missing, the honest answer is "not in the notes."

Working with the context window

The agent only uses what fits in the current conversation, plus the files it reads.

Include

- The one task and what done looks like
- The error message, pasted in full
- The file that matters (notes.md, index.html)

Exclude / start fresh

- Whole-repo dumps and old failed threads
- Secrets, .env, huge PDFs
- New chat when the agent is looping or confused

A clean prompt in a new session beats a messy 80-message thread. Commands: `/context` to inspect, `/clear` to reset, `/compact` to shrink. Corrected the same thing twice? Clear and write one better prompt.

Reviewing code at team scale

You already have a personal checklist from Day 1. This is what changes once a team is shipping fast with agents.

Before agents

- Fewer, slower pull requests
- Review: style, logic, naming
- Author remembers why each line exists

With agents

- PR volume goes up, lots of small diffs
- Review: did it actually run? secrets? extra files?
- Author must still explain the change

Watch for: silent dependency adds, deleted tests, "drive-by" refactors you didn't ask for.

Best practices and common mistakes

Do

- One small step at a time
- Test as you go: run it, click it
- Read the diff before accepting
- Commit working states

Don't

- Build the whole app in one prompt
- Trust output blindly, since it can be wrong
- Paste secrets or private keys
- Ship code you never ran

You are the engineer. The agent is a fast intern.

• WHERE TO GO NEXT

Roadmap: Machine Learning Engineer

This path goes deeper, and it needs real math. Here's the order that works.

1. Programming

- Python fundamentals
- Data structures & algorithms

2. Math foundations

- Linear algebra
- Calculus
- Probability & statistics

3. ML fundamentals

- Classical ML (scikit-learn)
- Then deep learning

4. Build & practice

Real projects, Kaggle competitions, reproduce papers.

Recommended math courses

3Blue1Brown (YouTube, free), Khan Academy (free), MIT 18.06 Linear Algebra (OCW, free), Andrew Ng's ML Specialization (Coursera).



Questions? DM on
LinkedIn

[linkedin.com/in/yusufmorsi](https://www.linkedin.com/in/yusufmorsi)



Or message on
WhatsApp

+20 122 262 1488

• WHERE TO GO NEXT

Roadmap: Software Engineer using AI

A faster path if you want to build with AI, not research it. No heavy math required.

1. SWE fundamentals

- Programming, git, web basics
- Data structures, just enough

2. Use AI tools well

- Prompt engineering
- Agentic coding (what we did!)

3. Integrate AI

- Call LLM APIs
- Practical RAG, no deep math

4. Ship real projects

Real apps, hackathons, a portfolio. Exactly the path these two days put you on.



Questions? DM on
LinkedIn

[linkedin.com/in/yusufmorsi](https://www.linkedin.com/in/yusufmorsi)



Or message on
WhatsApp

+20 122 262 1488

• WHERE TO GO NEXT

Research, mentors, and a reality check

The roadmap gets you skills. This is how you actually use them.

Do research, even remotely

Email professors, anywhere in the world, not just Egypt. Ask to volunteer for free in their lab, remotely. It builds your resume and gives you real hands-on experience you'll actually learn from.

Find mentors

I emailed professors and people working at Amazon and Google, just to meet and ask for advice. Mentorship matters. Most people say yes more often than you'd expect.

A reality check on the job market

It's very tough right now, in Egypt and abroad. Even with a referral, your odds are extremely low. I personally never got a job through a referral, but I know people who have, it does work, just don't expect it to be easy. Apply anyway.



Questions? DM on
LinkedIn

[linkedin.com/in/yusufmorsi](https://www.linkedin.com/in/yusufmorsi)



Or message on
WhatsApp

+20 122 262 1488

• DAY 3 • BUILD SOMETHING REAL

Hackathon

Same loop from yesterday. Open theme, bring your own idea.

Format

- Teams of 2-4 students
- No team? We'll help you form one
- Venue/timing announced soon

Resources

- Free AI credits from sponsor
- Claude Code, any language
- Public APIs OK

Any level welcome

- Frontend-only is welcome
- Full-stack/RAG scores bonus
- Pick a finishable scope

• THAT IS A WRAP

Thank you

Questions? Stay curious. Ship small. Read the diff.

Q&A now

Hands-on help after the talk

See you at the hackathon

• DAY 2 WRAP UP

What to expect on Day 3

Date and venue still to be announced. This is the hackathon, open theme, bring your own idea.

Format

Teams of 2 to 4. Exact hours and venue coming soon.

Resources

Free AI credits from our sponsor.

No team yet? We'll help you find one when you arrive. Exact date and venue will be announced through the ACM group and page once confirmed.